

COMPUTER SCIENCE TRIPOS Part IB – 2023 – Paper 4

6 Programming in C and C++ (djc11+am21)

Memory
allocation

- (a) Describe the two main forms of dynamic storage in languages such as C and OCaml? How does each potentially interact with a garbage collector? [4 marks]

Answer: The stack is managed by a simple pointer and requires de-allocation to happen in the reverse order w.r.t. allocation. The heap allows flexible release of regions, leading to fragmentation. It typically uses linked lists of blocks of various (e.g. Fibonacci) sizes. Only the heap is freed by the garbage collector, but both need to be scanned for pointers to in-use blocks.

- (b) Does any aspect of C or C++ prevent a compiler from implementing the ‘tail-recursion optimisation’ exploited in functional languages and would this tend to reduce memory leak problems (e.g. a web server running out of memory)? [2 marks]

Answer: A large number of tail calls could be optimised by the C compiler and a custom order of argument evaluation could help optimise a tail call according to data dependencies. Where addresses of local variables are passed to inner calls or used in other ways, the compiler should detect whether the parent stack frame needs to be maintained. In C++, exception handling and destructor side-effects may also raise obstacles. [Full credit is also available for any sensible counter argument.] The memory leak problem manifests as a full heap (out-of-memory) in a long-running server. Stack use is likely to be a function of the particular request being served and will be fully reclaimed before the next work item is handled by that thread, so how long the program has been running is typically irrelevant. Hence stack optimisations will not help with ‘leaks’.

- (c) An ‘in-core’ database system uses a number of client processes that share a segment of memory with a server process which manages data storage. Assuming there are currently four client processes running, describe or compute the total number of user-space segments likely to be allocated in physical memory. How might memory in the shared segment be allocated? [4 marks]

Answer: Simplistically, each client and server will have their own heap, stack, statics and code segments. This would nominally make $4 \times 5 = 20$ conventional segments with a further segment being shared. Total 21. However, if the four servers have the same code, their text segment (and any read-only static segments which we shall disregard) will be stored only once, taking off 3, giving 18. Statics, such as shared pointers and mutexes, in the shared memory might be allocated as fields of a struct nominally instantiated at the start of the region. Some of these statics would be used for management of the remainder of the segment as a shared heap. [Full credit for any reasonable, self-consistent answer.]

uses of virtual:
inheritance and
functions

- (d) C++ allows multiple inheritance, e.g.

```
class D: B,C { int d1, d2; };
```

where classes B and C have previously been defined. [Note: Your answer need not consider access qualifiers nor member functions.]

- (i) First assume that classes B and C do not inherit from other classes. Give C code which summarises the storage layout of a variable x of type `class D`. [3 marks]

Answer:

```
struct D { struct B t1; struct C t2; int d1, d2; };
```

where names `t1` and `t2` do not occur elsewhere in the C++ program.

Not needed for answer: suppose `c` is a member of class C then C++ code `x.c` would be represented in C as `x.t2.c`.

- (ii) Given `class D *p`, how can C++-style “cast to pointer to base class”, exemplified by `(B *)p` and `(C *)p`, be achieved by C code? [2 marks]

Answer: Using the above C declaration of D, the code would essentially be `&p->t1` or `&p->t2`. Alternative code for the first case (exploiting the fact that `t1` is at offset zero within class D) is `(struct B *)p` in C. Note that the second generally requires an addition of a constant (representing `sizeof B`) unlike Java. It means that `(struct C *)p` is *erroneous* C code. Additional subtlety [not needed for answer] for the second case: C++ upcasts are required to allow for `p` to be NULL, more correct C code would be `p==NULL ? NULL : &p->t2`

- (iii) Now suppose that classes B and C both inherit from class A. Explain a conceptual choice which arises as to how data members of class A appear within the storage layout for a variable x of type `class D`. [2 marks]

Answer: The issue is whether members `a1` and `a2` of class A should appear twice in D (once in B and once in C), or just once. This is visible to a programmer by, e.g., testing whether `x.B::a1` changes when `x.C::a1` is written to (assuming `a1` is a member of class A and `x` is a variable of type D).

- (iv) Explain how the `virtual` keyword allows a programmer to select between the alternatives in Part (d)(iii). [3 marks]

Answer: The C++ default behaviour is that both B and C get separate copies of A. Using `virtual` as in

```
class B: virtual A { int b1, b2};  
class C: virtual A { int c1, c2};  
class D: B,C { int d1, d2; };
```

results in a single copy of A being shared between the C and B classes inherited by D. A box-and-arrow diagram suffices here or the text below.

In C implementation terms (not required for full marks) we would have something like

```
struct B { struct A *t1p; int b1, b2};  
struct C { struct A *t2p; int c1, c2};  
struct D { struct A t0; struct B t1; struct C t2; int d1, d2; };
```

and an (implicit) constructor function for `D` would arrange for `x.t1.t1p = &x.t0` and `x.t1.t2p = &x.t0` to happen whenever an object `x` of class `D` is constructed. Note that classes `B` and `C` need to contain pointers to the shared instance of class `A` so that upcasts (from `D` to `B` or `C`) continue to work.
